This is because Elasticsearch employs an OR Boolean operator by default for this query, fetching all documents that match any of the search words. The words are matched individually rather than as a phrase: in our example, Elasticsearch searches for "Java" and returns the relevant documents, followed by another search for "Complete" that adds those results to the list, and so on. The query returns "Java", "Complete", or "Guide", including combinations of the words, as its results. The same search can be rewritten (although the OR operator is redundant) as the next listing shows.

Listing 10.6 Specifying the OR operator explicitly

```
GET books/_search
{
  "query": {
    "match": {
      "title": {
        "query": "Java Complete Guide",
        "operator": "OR"
      }
    }
  }
}
```

Specifies the OR operator explicitly (although it is set by default)

To change this behavior to find documents with all three words in the title, we need to enable the AND operator.

Listing 10.7 Specifying the AND operator explicitly

```
GET books/_search
{
  "query": {
    "match": {
      "title": {
        "query": "Java Complete Guide",
        "operator": "AND"
      }
    }
  }
}
```

Explicitly specifies an AND clause

This query tries to find a book or books that match all three words (the title must include "Java" *and* "Complete" *and* "Guide"). However, our data set does not contain a book called *Java Complete Guide*, so no results are returned.

10.5.5 Matching at least a few words

The OR and AND operators are opposing conditions. The OR condition fetches either of the search words, and the AND condition gets matching documents for exactly all of the words. What if we want to find documents matching at least a few words from the given set? In the previous example, suppose we want at least two words out of three to